Assignment # 3

Composition & Aggregation

■ General Instructions

- This assignment is to be submitted **individually**. Plagiarism of any form will result in a **zero** for all involved parties.
- You must use **C++** with object-oriented principles as taught in class. No STL containers (vector, list, map, etc.) are allowed.
- Your solution must compile and run without errors. Code that does not compile will receive **no marks**.
- Submit a single **.cpp** file named *RollNumber_Assignment3.cpp* on the LMS before the deadline.
- Late submissions attract a penalty of **10% per day**, up to a maximum of three days.
- Proper indentation, meaningful variable names, and inline comments are mandatory and are part of the marks.

■ Background

In this assignment you will model a simplified **Company Management System** using the OOP concepts of **Composition** (strong ownership — the composed object cannot exist independently) and **Aggregation** (weak ownership — the aggregated object has an independent lifecycle). You will implement the full class hierarchy shown below and wire them together inside a **Singleton** *Company* class that serves as the central management hub.

Q1 — Class Design & Prototypes [30 marks]

Design and implement the following **six classes**. Each class must have the prototype exactly as specified. You are free to add private helper methods, but the **public interface must not be altered**.

Name (Composition helper)

Encapsulates an employee's first and last name. This class is **composed inside Employee** — it cannot exist on its own.

```
class Name {
string f_name;
string l_name;
public:
Name(string f, string l);
```

```
void displayName(); // prints: Name: <f> <l>
};
```

Address (Composition helper)

Encapsulates an employee's permanent address (house number, block, street, city). Also **composed inside Employee**.

```
class Address {
int houseNo;
char block;
int streetNo;
string city;
public:
Address(int HN, char B, int SN, string C);
void displayAddress(); // prints: Address: HN-B streetNo.SN, city
};
```

Project (Aggregation)

Represents a company project. An employee is **aggregated** into a project — the employee continues to exist even if the project is dissolved. A project can have at most **10** employees.

```
class Project {
int ID;
string projectDescription;
int employeesWorkingOn;
Employee* employee[10];
public:
Project(int ID, string projectDescription);
int getID();
void displayProjectInfo();
void IncEmployeesWorkingOn();
void DecEmployeeWorkingOn();
bool AddEmployee(Employee* emp);
bool RemoveEmployee(Employee* emp);
void DisplayAllEmployees();
};
```

Department (Aggregation)

Represents a department. An employee is **aggregated** into a department — at most **50** employees per department. An employee can belong to **at most one** department at a time.

```
class Department {
int ID;
string name;
Employee* employee[50];
int employeeCount;
```

```
public:
Department(int ID, string name);
int getID();
void displayDeptInfo();
bool AddEmployee(Employee* emp);
bool RemoveEmployee(int employeeID);
void DisplayAllEmployees();
};
```

Employee (Composition + Aggregation target)

Core entity. Composes a *Name* and an *Address* object (strong ownership). Aggregates references to up to **3** Projects and **1** Department (weak ownership). Forward declarations of *Project* and *Department* are required.

```
class Employee {
int ID;
Name name; // Composition
Address permanentAddr; // Composition
bool assignedToDept;
Project* project[3]; // Aggregation
int projectCount;
Department* Dept; // Aggregation
public:
Employee(int ID, string f, string l,
int HN, char B, int SN, string city);
int getID();
void displayEmployeeInfo();
bool AddProject(Project* proj);
bool RemoveProject(int projectID);
void displayAllProjects();
void SetAssignedToDept(bool value);
bool GetAssignedTODept();
void SetDept(Department* dept);
void displayDept();
};
```

Company (Singleton)

Top-level manager. Only **one instance** can exist (Singleton pattern enforced via a static counter). Owns and manages up to 100 employees, 20 projects, and 4 departments via raw pointer arrays. All memory must be released in the destructor.

```
class Company {
static int count;
Employee* employees[100]; int ecount;
Project* projects[20]; int pcount;
Department* departments[4]; int dcount;
```

```
Company(); // private constructor
public:
static Company* construct(); // returns NULL if count >= 1
void createDepartment(int ID, string name);
void createProject(int ID, string description);
void createEmployee(int ID, string fn, string ln,
int HN, char B, int SN, string city);
void displayDeptID(int id);
void displayProjID(int id);
void displayEmpID(int id);
bool addEmpInDep(int idE, int idD);
bool addEmpInProj(int idE, int idP);
bool displayAllProj(int id);
bool displayAllEmpInDep(int id);
bool displayAllEmpInProj(int id);
bool displayDep(int id);
bool removeEmpFromDep(int idE, int idD);
bool removeEmpFromProj(int idE, int idP);
~Company();
};
```

Q2 — Relationships & Constraints [25 marks]

Implement the following behavioural constraints **exactly** as described. Your implementation will be tested with an automated harness.

One-Department Rule:

An employee can be assigned to **at most one department** at a time. Attempting to assign an already-assigned employee must print an informative error and leave state unchanged.

Three-Project Rule:

An employee can be associated with **at most three projects** simultaneously. Attempting to add a fourth project must return **false**.

Bidirectional Consistency:

When *Company::addEmpInDep()* is called, both the *Employee::SetDept()* and *Department::AddEmployee()* must be invoked to maintain consistency from both ends of the association.

Bidirectional Removal:

When *Company::removeEmpFromProj()* is called, both *Project::RemoveEmployee()* and *Employee::RemoveProject()* must be called.

Compact Arrays:

After every removal, shift remaining pointers left so there are no gaps in the middle of the array. The count variable must always reflect the actual number of elements.

Singleton Enforcement:

Company::construct() uses a static counter to guarantee only one *Company* object is ever created. A second call must return **NULL** and print an error.

Destructor:

The *Company* destructor must **delete** every heap-allocated object it owns (employees, projects, departments) to prevent memory leaks.

Q3 — Interactive Menu Driver [25 marks]

Implement a **main()** function that creates a single *Company* object and runs the following interactive menu in a loop until the user selects **0 — Exit**:

Option	Action
0	Exit
1	Create a new Department (ID + name)
2	Create a new Employee (ID + name + full address)
3	Create a new Project (ID + description)
4	Display a Department given its ID
5	Display an Employee given their ID
6	Display a Project given its ID
7	Add an Employee to a Department (both IDs)
8	Add an Employee to a Project (both IDs)
9	Display all Projects of an Employee (employee ID)
10	Display all Employees in a Department (department ID)
11	Remove an Employee from a Department (both IDs)
12	Remove an Employee from a Project (both IDs)
13	Display all Employees working on a Project (project ID)
14	Display the Department of an Employee (employee ID)

For invalid menu inputs print: "Invalid option. Please try again." and loop without changing any state.

■■ Academic Integrity & Plagiarism Policy

All submitted work must be your own. Copying code from classmates, online repositories, or AI-based tools (including but not limited to ChatGPT, GitHub Copilot, Gemini, Claude, or any code-generation service) constitutes plagiarism and is a direct violation of the FAST-NUCES Academic Integrity Policy.

The department employs both manual review and automated similarity-detection tools to identify plagiarised submissions. If two or more submissions are found to be substantially similar, **all involved students** will receive a **zero** for the assignment regardless of who is deemed the original author.

AI-generated code has recognisable stylistic and structural patterns. Submitting such code — even after superficial modification — will be treated as plagiarism. Students found in violation may additionally face disciplinary proceedings under the university's code of conduct, which can result in course failure or expulsion.

By submitting this assignment you confirm that the work is entirely your own and that you have neither given nor received unauthorised assistance.